

CSE 318 Assignment-03

Adversarial Search: Playing Mancala with Minimax and Alpha-Beta Pruning

Student ID: 2205029

September 2, 2026

1 Introduction

This report presents the implementation and experimental evaluation of a **Mancala**-playing agent based on the *Minimax* algorithm with *Alpha-Beta Pruning*. Four evaluation heuristics of increasing sophistication were implemented, and the agent’s move quality was measured by running a full computer-vs-computer tournament across every combination of heuristic and search depth for both players. The resulting win/loss statistics are summarized, tabulated, and visualized below, and are used to draw conclusions about which heuristics and search depths are most effective for this game.

1.1 Experimental Settings

- **Board:** the standard 6-pit-per-side Mancala board, 4 stones per pit, empty stores, Player 1 to move first.
- **Heuristics tested:** $\{1, 2, 3, 4\}$ (defined in Section 2.3), with all weights fixed at $W_1 = W_2 = W_3 = W_4 = 1$.
- **Search depths tested:** $\{2, 3, 4, 5, 6\}$ plies.
- **Tournament design:** every combination of (Player 1 heuristic, Player 1 depth, Player 2 heuristic, Player 2 depth) was played once, giving $4 \times 5 \times 4 \times 5 = 400$ games in total. Both players always use Minimax with Alpha-Beta pruning; only the heuristic and the depth limit differ between them.
- **Extra turns:** Mancala’s “land in your own store, go again” rule is respected inside the search tree itself – when a simulated move grants an extra turn, the search recurses into the *same* maximizing/minimizing function for the *same* player, rather than switching to the opponent.

2 Algorithm Descriptions

2.1 Mancala Rules Implemented

Each side has 6 pits (4 stones each at the start) and one store. A move consists of picking a non-empty pit on the mover’s own side and sowing its stones one at a time, counter-clockwise, into the following pits – skipping the opponent’s store but including the mover’s own store. Two special rules are implemented:

- **Extra turn:** if the last stone sown lands in the mover’s own store, the same player moves again.
- **Capture:** if the last stone sown lands in a previously empty pit on the mover’s own side, the mover captures that stone together with all stones in the directly opposite pit on the opponent’s side, moving them all to the mover’s store.

When one side’s pits are all empty, the game ends and all remaining stones on the other side are swept into that other player’s store.

2.2 Minimax with Alpha-Beta Pruning

Player 1 is treated as the maximizer and Player 2 as the minimizer; the evaluation function is always computed from Player 1’s perspective, so the maximizer seeks the highest value and the minimizer seeks the lowest. At each node, **alpha** tracks the best value the maximizer can already guarantee and **beta** tracks the best value the minimizer can already guarantee; a branch is pruned as soon as $\alpha \geq \beta$, since no player would ever allow the game to reach a node that is already worse for them than an alternative they have already secured elsewhere in the tree. Search is depth-limited: once the depth budget is exhausted (or the game ends), the static heuristic evaluation is used in place of continued search.

2.3 Heuristics

Let S_m, S_o denote the mover’s and opponent’s stored stones, and B_m, B_o denote the total stones remaining in the mover’s and opponent’s six pits, respectively.

$$H_1 = S_m - S_o \tag{1}$$

$$H_2 = W_1(S_m - S_o) + W_2(B_m - B_o) \tag{2}$$

$$H_3 = H_2 + W_3 \cdot (\text{extra-turn bonus}), \quad \text{extra-turn bonus} = \begin{cases} +1 & \text{I just earned an extra turn} \\ -1 & \text{opponent just did} \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

$$H_4 = H_3 + W_4 \cdot (\text{signed stones captured on the last move}) \tag{4}$$

H_1 rewards only banked stones; H_2 additionally values board material; H_3 further rewards positions where the side to move has just earned (or denied) an extra turn; H_4 additionally rewards positions immediately following a capture. All experiments here use $W_1 = W_2 = W_3 = W_4 = 1$.

3 Overall Results Summary

Table 1 gives the tournament-wide outcome across all 400 games. Table 2 and Table 3 break the win rate down by heuristic and by search depth respectively, combining each configuration’s results whether it was used as Player 1 or Player 2 (each heuristic and each depth therefore appears in exactly 200 and 160 games respectively).

Table 1: Overall tournament outcome across all 400 games.

Outcome	Games	Percentage
Player 1 wins	181	45.25%
Player 2 wins	201	50.25%
Draws	18	4.50%
Average Player 1 store		23.86
Average Player 2 store		24.14

Table 2: Win rate by heuristic (200 games each, combining Player 1 and Player 2 roles). Draws count as half a win.

Heuristic	Win Rate	Avg. Margin (own – opp. store)	Win Rate at Depth 6 only
H_1	64.50%	+6.69	86.2%
H_2	41.75%	−3.73	53.8%
H_3	45.50%	−1.79	60.0%
H_4	48.25%	−1.17	70.0%

Table 3: Win rate by search depth (160 games each, combining Player 1 and Player 2 roles). Draws count as half a win.

Depth	Win Rate	Avg. Margin (own – opp. store)
2	30.31%	−9.93
3	35.94%	−3.23
4	54.69%	+0.79
5	61.56%	+5.06
6	67.50%	+7.30

Two additional aggregate statistics are worth noting. When the two players search to *different* depths, the deeper-searching side wins 70.0% of those 320 games (224/320); when both players search to the *same* depth (the remaining 80 games), Player 1 wins only 43.75% of the time (35/80), suggesting no strong first-move advantage once search depth is equalized.

4 Graphical Comparison

Figure 1 shows the win rate achieved by each heuristic, and Figure 2 shows the win rate achieved at each search depth, both aggregated over all 400 games as in Tables 2 and 3. Figure 3 isolates the heuristic comparison to only the 40 games per heuristic played at the maximum depth of 6, showing that H_1 ’s advantage over the other heuristics grows even larger with deeper search.

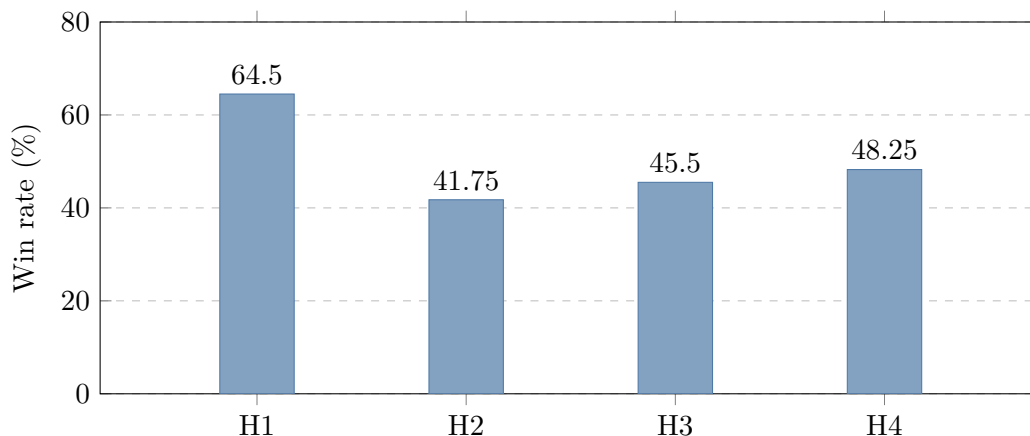


Figure 1: Win rate by heuristic, aggregated over 200 games each.

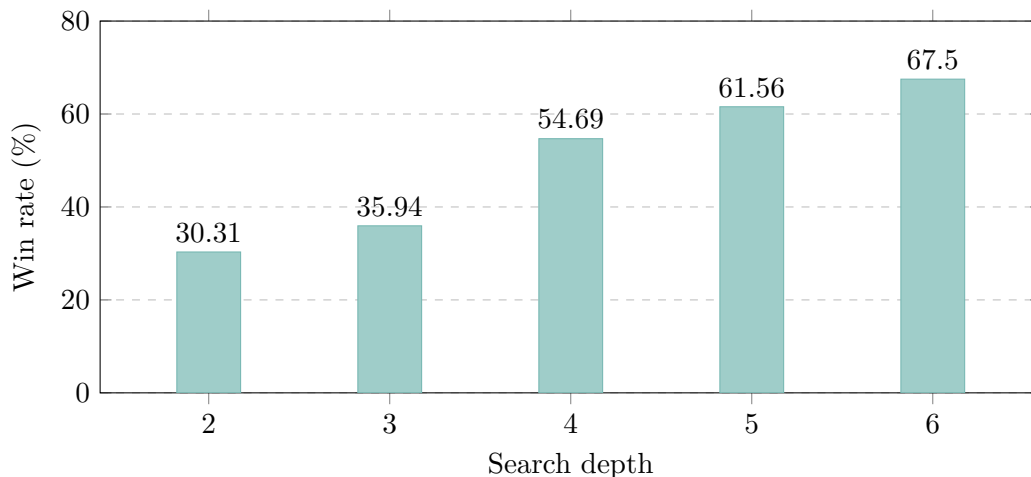


Figure 2: Win rate by search depth, aggregated over 160 games each.

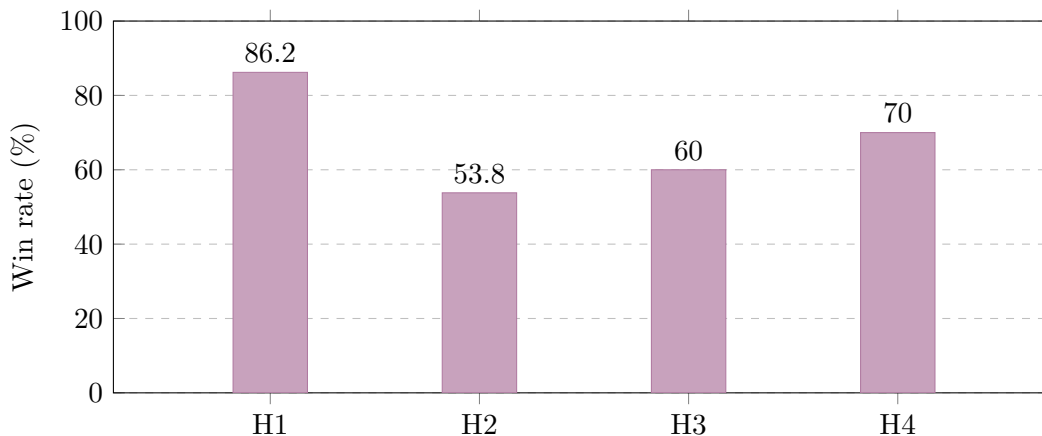


Figure 3: Win rate by heuristic, restricted to the 40 games per heuristic played at depth 6.

5 Observations

- **Search depth is the dominant factor.** Win rate climbs monotonically and substantially with depth, from 30.3% at depth 2 to 67.5% at depth 6 (Table 3, Figure 2), regardless of which heuristic is paired with it. When the two players search to different depths, the deeper side wins 70% of the time, confirming that additional lookahead is the single most reliable way to improve play in this implementation.
- **The simplest heuristic, H_1 , performs best overall.** Despite using none of the extra terms in H_2 – H_4 , H_1 achieves the highest win rate (64.5%) and the only positive average score margin (+6.69) among all four heuristics (Table 2). Its advantage widens further at the deepest search level tested: at depth 6, H_1 wins 86.2% of its games, far ahead of H_2 (53.8%), H_3 (60.0%), and H_4 (70.0%) (Figure 3). This indicates that, with equal ($W = 1$) weighting, the extra board-material, extra-turn, and capture terms add noise to the evaluation rather than useful signal – the search performs best when it optimizes directly for the quantity that decides the game (banked stones) rather than a proxy for it.

- **H_2 is the weakest of the four.** Adding the raw board-material term ($B_m - B_o$) with equal weight to the store-difference term actually reduces performance below H_1 alone (41.75% vs. 64.50%), and H_2 has the most negative average margin of any heuristic (-3.73). Having many stones on one’s own side is not intrinsically good – what matters is *which* pits they occupy and whether they are close to converting into an extra turn or a capture – so an equally-weighted raw-count term can actively mislead the search.
- **H_3 and H_4 partially recover the lost ground.** Adding the extra-turn bonus (H_3) and then the capture bonus (H_4) on top of H_2 steadily improves win rate (41.75% $\rightarrow$ 45.50% $\rightarrow$ 48.25%), suggesting these two additional signals are individually useful, but not enough on their own (at equal weight) to outweigh the harm done by the $B_m - B_o$ term they are built on top of.
- **Head-to-head, H_1 beats every other heuristic when both play optimally.** Restricting to games where Player 1 used H_1 , its win rate against opponents using H_1 , H_2 , H_3 , and H_4 respectively was 72.0%, 68.0%, 60.0%, and 68.0% – it does not simply win because of an easy schedule, but wins directly against each of the more elaborate heuristics.
- **Player 2 has a small overall edge, but it disappears at equal depth.** Player 2 won more games overall (201 vs. 181), but when both sides search to the same depth, Player 1’s win rate is only 43.75%, close to an even split. The overall Player 2 edge is therefore better explained by the interaction of heuristic/depth pairings across the 400-game schedule than by any inherent second-mover advantage in this implementation.
- **Weights were never tuned.** Every heuristic here uses $W_1 = \dots = W_4 = 1$, as specified in the assignment; the relative under-performance of H_2 – H_4 could plausibly be reversed with properly tuned (and likely much smaller) weights on the material, extra-turn, and capture terms, since those terms are individually informative but currently overweighted relative to the dominant store-difference term.

6 Conclusion

The tournament results show that, in this implementation, search depth matters far more than heuristic sophistication: win rate rises steadily and substantially from depth 2 to depth 6 regardless of which evaluation function is used, while the simplest heuristic (H_1 , pure store difference) consistently outperforms the more elaborate heuristics H_2 , H_3 , and H_4 under equal ($W = 1$) weighting, both overall and specifically at the deepest search level tested. This suggests that the additional heuristic terms – board material, extra-turn bonus, and capture bonus – carry real signal (since H_3 and H_4 each improve on H_2), but that the raw, equally-weighted material term in H_2 introduces enough noise to make the combined heuristic worse than the simplest possible one. A natural extension of this work would be to tune the weights W_1 – W_4 (for example via self-play, as outlined in the future-work discussion) rather than fixing them all at 1, which would likely let the richer heuristics realize the benefit their individual terms already show in isolation.